

Vertical Evolution Inherits Paper 1’s Orchestration-Rule Evolution

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 14, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work formalizes an inheritance edge between Paper 2 (Li, April 2026) and Paper 1 (Li, April 2026) of the Coordination Knowledge Substrate (CKS) theory series. It does not introduce new axioms. Its sole contribution is to show, in operational form, that Paper 2’s vertical evolution mechanism is an upward-directional extension of Paper 1’s orchestration-rule evolution pattern — not a novel mechanism that Paper 1 failed to anticipate.

Abstract

Paper 2 of the Coordination Knowledge Substrate (CKS) theory series introduces vertical evolution as the axis along which structural improvements propagate upward through the level hierarchy: cell-level insights can be proposed for uptake at aspect level, and aspect-level insights for uptake at Self level, with governance authorization required at each receiving level. This note establishes the inheritance edge C1.22: vertical evolution (Paper 2) $\supset$ orchestration-rule evolution (Paper 1). Four properties of Paper 1’s orchestration-rule evolution pattern are preserved without modification — governance authorization of each update, non-specialist applicability of the modify right, attribution of each update in the receiving entity’s lineage, and the pattern that rules evolve from accumulated operational experience. Four properties are new in Paper 2: the naming of vertical direction as a distinct mechanism, the multi-level propagation path with review at each level traversed, the positioning of the cell as the initiator of higher-level architectural reflection, and the goal of improving coordination architecture rather than simply updating individual rules. Together with C1.21, which establishes that horizontal evolution inherits Paper 1’s substrate content modifiability, this note closes the bidirectional evolution inheritance coverage for the CKS theory series.

1. The Inheritance Edge: Statement of the Claim

Paper 1 of the CKS series commits to orchestration rules as modifiable substrate content. Claim 3 and commitment A0.03 establish that governance practitioners hold a modify right over orchestration rules at any time, not only at deployment. This modify right enables orchestration-rule evolution: over the lifecycle of a deployment, governance accumulates experience about which coordination patterns work and which do not, and it updates orchestration rules to reflect that learning. Paper 1 further establishes, through the non-specialist governance principle (A1.11), that the modify right is exercisable by governance practitioners who operate the system — not only by the architects who designed it. The

practitioner who notices that a rule is not working is the same practitioner who can update it, provided they hold governance authority.

Paper 2 names vertical evolution as the mechanism by which improvements propagate upward through the three-level hierarchy (cell, aspect, Self). The mechanism is directional: cell-level insights can trigger governance review at aspect scope, potentially producing aspect-DNA updates; aspect-level insights can trigger governance review at Self scope, potentially producing Self-DNA updates. Each upward step requires a separate governance-authorized directed-selection event at the receiving level. The mechanism is structural: vertical evolution targets the composition architecture — the orchestration substrates that define how cells relate to aspects and how aspects relate to the Self — not only the DNA content within existing structure, which horizontal evolution covers.

The inheritance edge C1.22 states: vertical evolution is the upward-directional, multi-level extension of Paper 1's orchestration-rule evolution pattern. Every vertical evolution event is a governance-authorized update to coordination architecture, driven by accumulated operational experience, exercisable by non-specialist practitioners, and attributed in the receiving entity's lineage chain — all four of these are the Paper 1 orchestration-rule evolution pattern, applied at each level the improvement traverses.

2. The Inherited Side: Paper 1's Orchestration-Rule Evolution Pattern

To establish the inheritance, it is necessary to state precisely what Paper 1 commits to in the orchestration-rule evolution context.

Paper 1 locates orchestration rules as substrate content — the same kind of governed artifact as coordination entries, conflict records, and decision rationale. Rules are not frozen at deployment; they are modifiable under the same authority architecture that governs all substrate content. The modify right is the mechanism: a governance practitioner who holds modify authority over a cell's orchestration rules can update those rules at any time, for any reason that governance deems warranted.

Paper 1 further commits that orchestration-rule evolution is experience-driven. The canonical case is a governance practitioner observing, through operation, that a coordination pattern is producing suboptimal outcomes — escalating too frequently, routing conflicts to the wrong handler, generating substrate entries that downstream cells cannot parse — and updating the rule to correct the pattern. The trigger for the update is accumulated operational experience, not a design-time decision. Rules are expected to improve over time as the system is actually used.

The non-specialist governance principle adds that this experience-driven modify right is not reserved for architects. A practitioner who has been operating the system for months and who has observed a coordination failure firsthand can update the responsible rule without needing to understand the rule's original design rationale. The architecture does not require that governance and authorship roles be held by the same humans. This is what makes orchestration-rule evolution a property of practical deployments, not only of deployments operated by their original designers.

Attribution is the final element. Each orchestration-rule update is recorded in the substrate's lineage — who made the change, when, and against what prior version. This attribution supports path retraceability: an observer can reconstruct the full history of how a rule evolved from its initial form to its current form.

These four properties — governance authorization, non-specialist applicability, experience-driven trigger, and lineage attribution — together define the orchestration-rule evolution pattern that Paper 1 commits to.

3. The Extending Side: Paper 2's Vertical Evolution

Paper 2 adds three levels of architectural composition: cells, aspects, and Selves. At each level, the same CKS commitments from Paper 1 apply by inheritance — the same modify right, the same non-specialist governance principle, the same attribution requirements. What Paper 2 introduces is a named mechanism for improvement propagation across these levels in the upward direction.

Vertical evolution operates as follows. A cell-level operational insight — surfaced through a directed-selection event, an action-feedback proposal, or a practitioner observation — may have applicability beyond the cell where it was first noticed. If the insight reveals a coordination pattern that is suboptimal not just for that cell but for how cells relate within their aspect, the insight can be proposed for uptake at aspect level. Governance at aspect scope reviews whether the insight is relevant at that scope. If it is, the aspect DNA is updated through a governance-authorized directed-selection event at aspect scope. The update is recorded in the aspect's lineage chain. If the aspect-level insight further has applicability at Self scope, governance at Self scope reviews whether the insight is relevant there. If warranted, the Self DNA is updated through a governance-authorized directed-selection event at Self scope, recorded in the Self's lineage chain.

The key structural constraint of vertical evolution is that it cannot skip levels. An insight that originates at cell scope and is relevant at Self scope must still traverse aspect-scope governance review before reaching Self scope. The level-by-level constraint is not bureaucratic overhead; it is the architectural mechanism that ensures higher-level coordination architecture is not rewritten by cell-level operational findings without intermediate review. Cell-level observations are necessarily local — they reflect the operational experience of one cell in one context. Aspect-scope governance review is what determines whether the local observation generalizes to the broader coordination arrangement. Self-scope governance review is what determines whether the aspect-level pattern generalizes to the integration architecture. Each level of review is doing architectural work that the levels below it cannot do.

This level-by-level constraint means that vertical evolution cannot produce automatic upward propagation. A cell-level insight, however compelling, does not update Self-level architecture on its own. It initiates a proposal that must be reviewed and accepted at each intermediate level before reaching any higher level. The governance authorization at each level is independent — the aspect-level review does not obligate the Self-level review to accept the same insight.

4. What Is Preserved: The Orchestration-Rule Evolution Identity

Four properties of Paper 1's orchestration-rule evolution pattern carry through unchanged into vertical evolution.

Governance authorization at each level. Every vertical evolution step is a governance-authorized directed-selection event at the receiving level. The receiving level's governance practitioners hold modify authority over the receiving entity's DNA, and they exercise that authority when they approve the update. This is precisely the modify right and orchestration-rule evolution pattern from Paper 1: governance practitioners with modify authority update coordination rules based on operational experience. Vertical evolution does not introduce a new kind of authorization; it applies Paper 1's governance-authorized update at each level the improvement traverses.

Non-specialist governance applicability. The practitioner who observed the cell-level insight can initiate the vertical evolution proposal. They do not need architect authority over the aspect or Self levels. What they need is governance authority at the cell level — which, by Paper 1's non-specialist governance principle, is available to any practitioner who operates the system. The review at each higher level is performed by governance practitioners at that level's scope, who also need not be architects of the level above them. The accessibility of the modify right that Paper 1 commits to applies at every level of the vertical evolution propagation path.

Attribution in each receiving entity's lineage. Each vertical evolution event produces a lineage entry in the receiving entity's record. When a cell-level insight triggers an aspect-DNA update, that update appears in the aspect's lineage chain. When an aspect-level insight triggers a Self-DNA update, that update appears in the Self's lineage chain. The cell-level insight that originally initiated the chain is traceable through the proposal and review records at each level. Paper 1's attribution requirement — that every governed substrate update is recorded with authorship, timestamp, and prior-version reference — applies without modification at each level vertical evolution touches.

Rules updated from accumulated experience. Vertical evolution is explicitly triggered by operational experience at cell level. The cell-level insight arises from the cell being operated — from action-feedback accumulation, from practitioner observation of coordination failures, from directed-selection events identifying suboptimal patterns. The trigger is not a design-time decision; it is the discovery, through use, that the current coordination architecture is not working as well as it could. This is Paper 1's orchestration-rule evolution pattern: rules evolve from accumulated experience, not from architectural planning alone. Vertical evolution extends this pattern to the level hierarchy — the experience accumulates at cell scope, and governance reviews its relevance at each higher scope.

5. What Is New in Paper 2

Four properties of vertical evolution are genuinely new relative to Paper 1's orchestration-rule evolution pattern.

Vertical direction as a named mechanism. Paper 1 had orchestration-rule evolution at one scope — the cell level — with no named direction of propagation. Paper 2 names the upward direction as a distinct mechanism: vertical evolution. The naming is not cosmetic. By naming the direction, Paper 2 makes it possible to specify the multi-level propagation path, to distinguish vertical from horizontal

evolution, and to reason precisely about what governance authorization is required at each level. Without the named mechanism, the directionality of improvement propagation would be implicit and unspecifiable.

Multi-level propagation path with review at each level. Paper 1's orchestration-rule evolution operates within a single cell — a practitioner updates a rule, the update takes effect in that cell. Paper 2's vertical evolution traverses a defined propagation path: cell insight → aspect review → aspect update → Self review → Self update. Each step in the path is a distinct governed event. The path is bounded — it begins at cell scope and terminates when either a level's governance review finds the insight not warranted or the Self level has been updated. The multi-level path with independent review at each level is a structural extension with no counterpart in Paper 1.

Cell as initiator of higher-level architectural reflection. In Paper 1, orchestration-rule evolution can be initiated by any governance practitioner with modify authority over the relevant rule. Paper 2's vertical evolution specifically positions the cell-level operational context as the trigger for higher-level architectural reflection. The insight arises from cell-level operation; the proposal moves upward from there. The bottom-up trigger direction — cell operations driving governance review of aspect and Self architecture — is a new architectural commitment. It is not that Paper 1 prohibited bottom-up triggers; Paper 1 simply had no level hierarchy to define a direction within. Paper 2 introduces the hierarchy and names the upward direction as the operational channel through which cell-level experience reaches coordination architecture.

Architectural coherence as goal. Paper 1's orchestration-rule evolution is oriented toward correcting individual rules — making a specific coordination pattern work better in a specific cell. Paper 2's vertical evolution is oriented toward improving coordination architecture: the DNA that defines how aspects organize cells and how the Self integrates aspects. The goal is not to fix one rule but to propagate a pattern of improvement upward through the level hierarchy so that the coordination architecture as a whole reflects what operational experience has revealed. This architectural coherence goal — maintaining alignment between cell-level operational reality and the higher-level structural commitments that govern cell behavior — is a new purpose that has no counterpart in Paper 1's single-level orchestration-rule evolution.

6. The Bidirectional Evolution Pair: C1.21 and C1.22

This note is the second in a two-note set that together close the bidirectional evolution inheritance coverage for the CKS series.

C1.21 establishes that horizontal evolution inherits Paper 1's substrate content modifiability. Horizontal evolution refines content within existing structure: cells refine their DNA, aspects refine their cell composition, Selves refine their aspect arrangement. The improvement stays within the level where it originates — a cell's DNA update does not require governance review at aspect or Self scope; it is a modification of content within the cell's own governed boundary. C1.21 shows that this is Paper 1's substrate content modifiability pattern applied at each level of the three-level hierarchy.

C1.22 (this note) establishes that vertical evolution inherits Paper 1’s orchestration-rule evolution pattern. Vertical evolution reorganizes structure: cell-level insights propagate upward to change aspect-level and Self-level coordination architecture. The improvement must traverse governance review at each level it crosses. C1.22 shows that each upward step is a governance-authorized directed-selection event applying Paper 1’s orchestration-rule evolution pattern at the receiving level.

Together, C1.21 and C1.22 cover all directional cases. Horizontal evolution covers the lateral direction: improvements spreading across peers within a level. Vertical evolution covers the upward direction: improvements propagating from lower levels to higher levels. Every architectural improvement a CKS deployment can make falls into one of these two categories — either it stays within the level where the insight originated (horizontal), or it propagates upward to change the composition architecture at a higher level (vertical). The bidirectional evolution pair shows that both categories are extensions of Paper 1 commitments, not novel mechanisms.

7. Prior-Art Significance

This note forecloses three classes of adversarial claim.

First, a claim that bottom-up architectural improvement — cell-level insights propagating to coordination architecture — is novel relative to Paper 1’s orchestration-rule evolution pattern. The claim fails because vertical evolution’s core mechanism is governance-authorized directed selection driven by accumulated operational experience, which is the Paper 1 orchestration-rule evolution pattern applied at each level the improvement traverses. The multi-level structure is new; the underlying mechanism is inherited.

Second, a claim that vertical propagation across governance levels is a novel mechanism beyond Paper 1’s scope. The claim fails because each vertical evolution step is individually a governance-authorized directed-selection event at the receiving level — applying Paper 1’s modify right and orchestration-rule evolution pattern at aspect scope and Self scope, with the same non-specialist applicability and attribution requirements. The path is new; each step on the path is inherited.

Third, a claim that cell-triggered higher-level governance reflection is novel relative to Paper 1’s operational-experience-based rule evolution. The claim fails because Paper 1 already commits to the pattern of governance updating rules based on operational experience observed at the cell level. Vertical evolution names the upward direction this experience can travel and adds the level-by-level constraint on how it travels; it does not introduce the experience-triggered update pattern.

8. Operational Test

A system implements vertical evolution as an inheritance of Paper 1’s orchestration-rule evolution pattern if and only if, for any vertical evolution event that has occurred in the system, an observer can do all of the following.

First, identify the cell-level insight that initiated the vertical evolution event: the specific operational experience — an action-feedback record, a practitioner observation, a directed-selection outcome —

that triggered the upward proposal.

Second, trace the governance review at each level the improvement traversed: find the aspect-scope governance review record showing that the insight was evaluated for relevance at aspect scope, and — if the improvement propagated to Self scope — find the Self-scope governance review record showing that the insight was evaluated at Self scope. If the improvement stopped at aspect scope, confirm that no Self-scope update was made without a corresponding Self-scope review.

Third, find separate attributed directed-selection records in each receiving entity’s lineage chain. The aspect’s lineage chain should contain an entry attributing the aspect-DNA update to the vertical evolution event. If the Self’s DNA was also updated, the Self’s lineage chain should contain a separate entry attributing the Self-DNA update to its own governance-authorized directed-selection event at Self scope.

A system in which a cell-level insight directly updates Self-level architecture without traversing aspect-scope governance review does not implement vertical evolution as specified by Paper 2 and does not preserve the level-by-level governance review constraint that this note identifies as the structural guard distinguishing vertical evolution from uncontrolled bottom-up propagation. A system in which the practitioner who observed the cell-level insight is excluded from initiating the vertical evolution proposal — because initiation requires architect authority rather than practitioner-level governance authority — does not preserve the non-specialist governance applicability that Paper 1 commits to and that vertical evolution inherits.

9. Conclusion

Paper 2’s vertical evolution mechanism is not independent of Paper 1’s orchestration-rule evolution pattern. It is the upward-directional, multi-level extension of that pattern. Governance authorization at each level, non-specialist applicability of the modify right, attribution in each receiving entity’s lineage, and the experience-driven trigger for updates all carry through from Paper 1 without modification. What Paper 2 adds is the naming of vertical direction as a distinct mechanism, the multi-level propagation path with level-by-level governance review as its structural constraint, the positioning of cell-level operations as the trigger for higher-level architectural reflection, and the goal of maintaining coordination architecture coherence across the level hierarchy.

Together with C1.21, which establishes the horizontal evolution inheritance, this note closes the bidirectional evolution coverage: the complete set of directional evolution moves a CKS deployment can make — across peers within a level and upward through the level hierarchy — derives from Paper 1 commitments through the inheritance edges C1.21 and C1.22 establish.